National University of Computing and Emerging Sciences (FAST NUCES)

Department of Computer Science / Data Science
Spring Semester 2026

Machine Learning Operations (MLOps)

Course Project

End-to-End MLOps Pipeline with Real-Time Monitoring, Alerting, and CI/CD

Automated Model Retraining · Prometheus · Grafana · Alertmanager · Slack · AWS · GitHub Actions

Course:	Machine Learning Operations (MLOps)
Semester:	Spring 2026
Total Marks:	100 Points + 10 Bonus Points
Submission:	As specified by the instructor

This project is designed to simulate a production-grade MLOps environment. Students are expected to integrate data pipelines, model lifecycle management, cloud deployment, and observability tooling into a single coherent system.

0. Contents

1. Project Overview

In modern machine learning engineering, shipping a model is only the beginning. Production systems must continuously monitor data quality, detect drift, respond to failures, retrain models automatically, and notify stakeholders — all without manual intervention. This project tasks you with building exactly such a system, end-to-end.

You will:

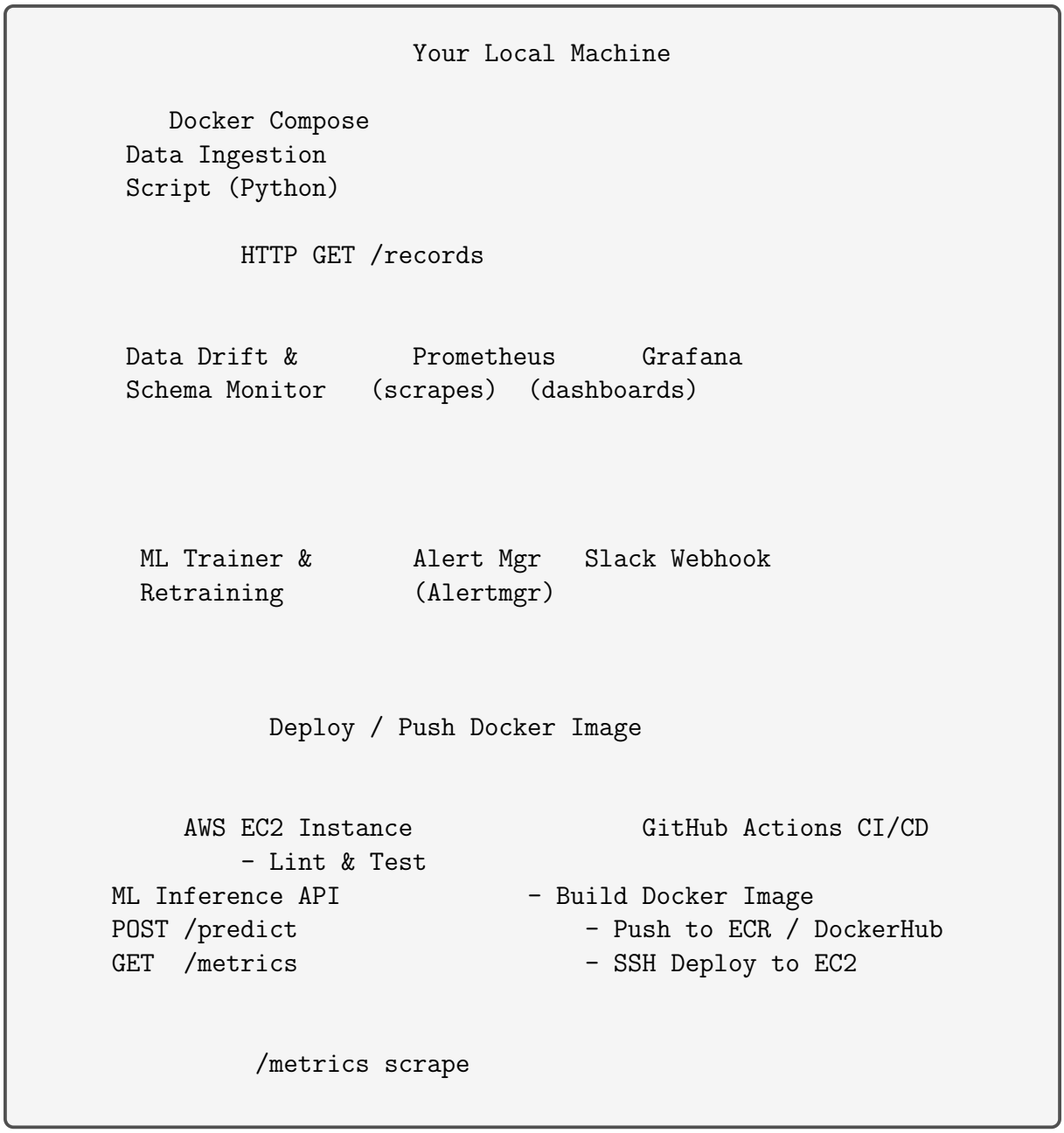
1. Ingest a **changing dataset** from a provided HTTP API endpoint that simulates real-world data evolution (schema changes, distribution shifts, intermittent unavailability).
2. Train a **machine learning classifier** and deploy it as a REST API on **AWS**.
3. Expose **Prometheus-compatible metrics** from your ML service.
4. Set up a full **observability stack** (Prometheus + Grafana + Alertmanager) using Docker Compose.
5. Configure **real-time Slack alerts** for critical system events.
6. Implement **automated model retraining and redeployment** when model quality degrades.
7. Automate your build, test, and deploy pipeline using **GitHub Actions CI/CD**.

Note

This project is intentionally scoped to require **8–9 hours** of focused work. Plan your time across the sections below. It is strongly recommended to complete tasks in the order presented, as later sections depend on earlier ones.

2. System Architecture Overview

The overall system you will build follows this high-level architecture:



3. Part 1 — Data Ingestion and Schema Monitoring

Estimated Time: 1.0 – 1.5 hours

3.1. Data Source

Rather than a fixed static dataset, this project uses a **live HTTP endpoint** that returns evolving batches of records. This simulates a real-world data pipeline where upstream data can change at any time — new features may appear, old features may disappear, and the service may become temporarily unavailable.

Important

You are permitted to call **only** the following endpoint:

`http://149.40.228.124:6500/records`

Calling any other endpoints on this server is not permitted and may result in disqualification of the relevant task.

3.2. API Behavior

- **Success (HTTP 200):** Returns a JSON payload containing:
  - `schema`: a list of feature names currently in the dataset.
  - `records`: a batch of data rows matching the current schema.
- **Service Unavailable (HTTP 503):** The endpoint is intentionally configured to return 503 errors at intervals, simulating data source downtime. Your system **must** detect this, log it, increment the appropriate Prometheus counter, and fire a Slack alert.

3.3. What Your Ingestion Script Must Do

Write a Python script or service (`ingestion.py` or similar) that:

1. Calls the `/records` endpoint at regular intervals (e.g., every 30–60 seconds or upon invocation).
2. **Parses and stores** each batch locally (e.g., appending to a CSV or Parquet file, or storing in memory for the session).
3. **Detects schema changes** by comparing the current batch's schema to the previously seen schema:
   - If a new feature appears: log it and fire the `feature_added` metric and Slack alert.
   - If an existing feature disappears: log it and fire the `feature_removed` metric and Slack alert.
4. **Detects distribution drift** by computing simple statistics (mean, std) per feature and comparing to a baseline. If drift exceeds a configurable threshold, fire the `distribution_drift_detected` metric and Slack alert.
5. **Handles 503 responses** gracefully: log the failure, increment `datalake_unavailable` counter, and fire a Slack alert.
6. Triggers the retraining pipeline if enough new data has accumulated or schema/drift changes are detected.

Deliverable

- `ingestion/ingestion.py` — data fetching and schema monitoring logic.
- `ingestion/drift_detector.py` — distribution drift detection logic.
- Brief inline comments explaining each decision.

4. Part 2 — ML Model Training and Auto-Retraining

Estimated Time: 1.0 – 1.5 hours

4.1. Training Requirements

Using the data ingested in Part 1, train a **binary or multi-class classifier**. The specific algorithm is your choice (logistic regression, random forest, gradient boosting, etc.), but the following conditions must be met:

- Training must run until **validation accuracy** $\geq$ **0.80**.
- If the target accuracy is not achieved within a configurable number of iterations, log a warning and proceed with the best achieved model.
- The trained model must be serialized (e.g., `joblib`, `pickle`, or ONNX) and stored with a versioned filename (e.g., `model_v1.pkl`, `model_v2.pkl`).
- Model accuracy and version must be recorded and exposed via the Prometheus exporter.

4.2. Auto-Retraining Logic

Your system must **automatically retrain** the model when any of the following conditions are detected:

1. **Accuracy drops below 0.80** — detected by comparing predictions on a held-out validation set with the latest ingested data.
2. **Significant distribution drift** is detected in the incoming data.
3. **Schema change** (feature added or removed) that requires re-engineering the feature pipeline.

When retraining occurs:

- Increment the `retrain_count_total` Prometheus counter.
- Log the reason for retraining.
- After successful retraining, redeploy the updated model to AWS (see Part 3).
- Send a Slack notification indicating retraining was triggered, the reason, and the new accuracy.

Deliverable

- `model/train.py` — training script with accuracy threshold logic.
- `model/retrain_trigger.py` — auto-retraining orchestration.
- `model/model_v{N}.pkl` (or equivalent) — serialized model artifact.

5. Part 3 — AWS Deployment

Estimated Time: 1.5 – 2.0 hours

5.1. Overview

Your ML inference service must be deployed and publicly reachable on AWS. You may use one of the following approaches (your choice):

- **AWS EC2** (recommended): Launch a free-tier EC2 instance, run your model server inside a Docker container.
- **AWS App Runner / ECS** (optional alternative): If you are comfortable with container orchestration services.

5.2. ML Inference API

Containerize your model server using Docker. The running container must expose:

Endpoint	Method	Description
`/predict`	POST	Accepts JSON feature input, returns prediction and confidence.
`/metrics`	GET	Returns Prometheus-format metrics (see Part 4).
`/health`	GET	Returns {`"status": "ok"`} for health checks.

5.3. Deployment Steps

1. Write a `Dockerfile` for your model server (FastAPI or Flask recommended).
2. Push the Docker image to **Docker Hub** or **AWS ECR**.
3. Launch an EC2 instance (Ubuntu 22.04, `t2.micro` or `t3.micro` is sufficient).
4. SSH into the instance and pull and run your Docker container.
5. Configure the EC2 Security Group to allow inbound traffic on the relevant ports (e.g., 80, 8000, 9090, 3000).
6. Record your EC2 public IP — it will be used to configure Prometheus scraping.

5.4. Deployment Script

Provide a shell script `deploy/deploy.sh` that automates the steps of building, pushing, and running the Docker image on the EC2 instance. This script will also be invoked by GitHub Actions (see Part 6).

Deliverable

- `Dockerfile` for the ML inference service.
- `deploy/deploy.sh` — automated deployment script.
- Screenshot of the running EC2 instance and a successful `curl` to `/health` and `/predict`.
- EC2 public IP documented in your `README.md`.

6. Part 4 — Prometheus Metrics Exporter

Estimated Time: 0.5 – 1.0 hour

6.1. Required Metrics

Your ML service must expose a `/metrics` endpoint in the standard **Prometheus text format**. You can use the `prometheus_client` Python library to implement this. The following metrics are **required**:

Metric Name	Type	Description
`model_accuracy`	Gauge	Current validation accuracy of the deployed model (0.0 – 1.0).
`records_processed_total`	Counter	Total number of records ingested from the API since startup.
`retrain_count_total`	Counter	Total number of times the model has been retrained.
`distribution_drift_detected`	Gauge	Set to 1 when drift is detected in the current batch, 0 otherwise.
`feature_added`	Counter	Number of features added to the schema since startup.
`feature_removed`	Counter	Number of features removed from the schema since startup.
`datalake_unavailable`	Counter	Number of times the `/records` endpoint returned 503.
`response_delay_seconds`	Histogram	Latency of each `/predict` API call in seconds.

6.2. Implementation Notes

- Use the `prometheus_client` library: `pip install prometheus-client`.
- Expose metrics at `http://<your-ec2-ip>/metrics` (or port 8000 if you configure it separately).
- Metrics must be **live** — they should reflect the real current state of the system, not mocked/static values.

Deliverable

- `exporter/metrics.py` or equivalent, defining all Prometheus metrics.
- Metrics exposed correctly at the `/metrics` endpoint on your AWS instance.
- Screenshot of the raw `/metrics` output in a browser or `curl`.

7. Part 5 — Prometheus, Grafana, and Alertmanager Stack

Estimated Time: 1.0 – 1.5 hours

7.1. Overview

You will deploy the full observability stack locally (or on a separate EC2 instance) using **Docker Compose**. The stack consists of:

- **Prometheus** — scrapes the `/metrics` endpoint on your AWS ML service.

- **Grafana** — visualizes the scraped metrics in dashboards.
- **Alertmanager** — receives alert rules from Prometheus and routes notifications to Slack.

7.2. docker-compose.yml Requirements

Your `docker-compose.yml` must define all three services. A minimal structure is shown below:

Listing 1: Minimal docker-compose.yml structure

```
version: "3.8"
services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.
        yml
      - ./prometheus/alert_rules.yml:/etc/prometheus/alert_rules
        .yml
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana

  alertmanager:
    image: prom/alertmanager:latest
    volumes:
      - ./alertmanager/alertmanager.yml:/etc/alertmanager/
        alertmanager.yml
    ports:
      - "9093:9093"

volumes:
  grafana_data:
```

7.3. Prometheus Configuration

Create `prometheus/prometheus.yml`. It must:

- Set the global scrape interval to 15 seconds.
- Configure a scrape job targeting your AWS EC2 instance's `/metrics` endpoint.
- Reference the `alert_rules.yml` file for alerting rules.

7.4. Grafana Dashboard

Configure Grafana with Prometheus as a data source and create a dashboard that visualizes:

- `model_accuracy` over time (line chart).
- `records_processed_total` rate (counter-rate graph).
- `retrain_count_total` (stat panel).
- `distribution_drift_detected` (status/indicator panel).
- `datalake_unavailable` total (stat panel).
- `response_delay_seconds` 95th-percentile (histogram quantile).

Export your dashboard as a JSON file and include it in `grafana/dashboards/`.

Deliverable

- `docker-compose.yml` — complete stack definition.
- `prometheus/prometheus.yml` — Prometheus configuration.
- `prometheus/alert_rules.yml` — alerting rules (see Part 6).
- `alertmanager/alertmanager.yml` — Alertmanager routing config.
- `grafana/dashboards/mlops_dashboard.json` — exported Grafana dashboard.
- Screenshots of the running Grafana dashboard with live data.

8. Part 6 — Alerting via Slack

Estimated Time: 0.5 – 1.0 hour

8.1. Overview

Alerts are the backbone of any production MLOps system. You will configure **Prometheus alerting rules** and route them through **Alertmanager** to a **Slack webhook**. All seven alerts below must be configured, fired, and demonstrated.

8.2. Required Alert Rules

Define the following alert rules in `prometheus/alert_rules.yml`:

#	Alert Name	Trigger Condition	Slack Message
1	`DataLakeUnavailable`	`increase(datalake_unavailable[1m]) > 0`	*"Data lake returned 503. Check API availability."*
2	`FeatureAdded`	`increase(feature_added[1m]) > 0`	*"New feature detected in schema. Retraining may be required."*

#	Alert Name	Trigger Condition	Slack Message
3	`FeatureRemoved`	`increase(feature_removed[1m]) > 0`	*"Feature dropped from schema. Verify pipeline compatibility."*
4	`DistributionDrift`	`distribution_drift_detected == 1`	*"Data distribution drift detected. Model may be stale."*
5	`FeatureDriftDetected`	`distribution_drift_detected > 0`	*"Feature-level drift flagged. Investigate upstream data."*
6	`HighResponseLatency`	`histogram_quantile(0.95, response_delay_seconds_bucket) > 1.0`	*"P95 response latency exceeded 1 second."*
7	`LowModelAccuracy`	`model_accuracy < 0.8`	*"Model accuracy dropped below threshold. Auto-retraining triggered."*

8.3. Alertmanager Slack Configuration

Configure `alertmanager/alertmanager.yml` to route all alerts to a Slack channel using a webhook URL. The webhook URL must be **externalized as an environment variable** (`SLACK_WEBHOOK_URL`) so it is not hardcoded in your repository.

Listing 2: Example Alertmanager Slack receiver snippet

```
receivers:
  - name: "slack-notifications"
    slack_configs:
      - api_url: "${SLACK_WEBHOOK_URL}"
        channel: "#mlops-alerts"
        send_resolved: true
        title: "[{{ .Status | toUpper }}] {{ .CommonLabels.
           alertname }}"
        text: "{{ range .Alerts }}{{ .Annotations.description
           }}{{ end }}"
```

Note

To demonstrate that alerts fire correctly, you may temporarily inject artificial conditions (e.g., set a metric value manually, or temporarily lower the accuracy threshold). Include screenshots of each Slack alert message received.

Deliverable

- `prometheus/alert_rules.yml` — all 7 alert rules defined.
- `alertmanager/alertmanager.yml` — Slack webhook routing config.
- Screenshots of **each alert firing** in your Slack channel.

9. Part 7 — GitHub Actions CI/CD Pipeline

Estimated Time: 1.0 – 1.5 hours

9.1. Overview

A professional MLOps workflow automates testing, building, and deploying your service. You will create a **GitHub Actions** workflow that runs on every push to the `main` branch and automatically deploys the updated model to AWS.

9.2. Workflow Requirements

Create `.github/workflows/mlops-ci.yml`. The workflow must include the following jobs/steps:

9.2.1. Job 1: Lint and Test

- Check out the repository.
- Set up Python 3.10+.
- Install dependencies from `requirements.txt`.
- Run `flake8` (or `ruff`) for code linting.
- Run `pytest` for any unit tests you have written (see Section 9.3).

9.2.2. Job 2: Build and Push Docker Image

- Build the Docker image for the ML inference service.
- Log in to Docker Hub (or AWS ECR) using repository secrets (`DOCKER_USERNAME`, `DOCKER_PASSWORD`).
- Push the tagged image (use `:latest` and `:<git-sha>` tags).

9.2.3. Job 3: Deploy to AWS EC2

- SSH into your EC2 instance using the private key stored in `EC2_SSH_KEY` repository secret.
- Pull the latest Docker image on the EC2 instance.
- Stop the currently running container and start the new one.
- Verify deployment by calling `/health` endpoint and asserting HTTP 200.

9.3. Required Unit Tests

Write at least **3 unit tests** in a `tests/` directory:

1. **Test schema change detection:** Simulate two batches with differing schemas and assert the correct feature_added / feature_removed flags are raised.
2. **Test drift detection:** Provide a reference distribution and a clearly shifted distribution; assert that drift is flagged.

3. **Test predict endpoint:** Mock the model and assert that `/predict` returns the expected JSON structure with a `prediction` key.

9.4. Repository Secrets

Configure the following secrets in your GitHub repository settings (*Settings ¿ Secrets and Variables ¿ Actions*):

Secret Name	Value
`DOCKER_USERNAME`	Your Docker Hub username
`DOCKER_PASSWORD`	Your Docker Hub access token
`EC2_SSH_KEY`	Private SSH key for EC2 instance (PEM content)
`EC2_HOST`	Public IP or DNS of EC2 instance
`EC2_USER`	SSH user (e.g., `ubuntu`)
`SLACK_WEBHOOK_URL`	Slack incoming webhook URL

Important

Never hardcode credentials in your code or `docker-compose.yml`. All sensitive values must be passed as environment variables or GitHub Secrets. Repositories with exposed credentials will lose marks on the Documentation rubric criterion.

Deliverable

- `.github/workflows/mlops-ci.yml` — complete workflow file.
- `tests/` directory with at least 3 passing unit tests.
- Screenshot of a **passing GitHub Actions workflow run** showing all jobs green.

10. Recommended Project Structure

Listing 3: Recommended repository layout

```
mlops-project/
          .github/
                workflows/
                    mlops-ci.yml            # GitHub Actions
   pipeline
          ingestion/
                ingestion.py                # Data fetching &
   schema monitoring
                drift_detector.py           # Distribution drift
   logic
          model/
                train.py                    # Training script
                retrain_trigger.py          # Auto-retraining
   orchestration
                model_v1.pkl                # Example serialized
   model
          serving/
```

```
              app.py                          # FastAPI / Flask
inference API
              Dockerfile                      # Container for ML
service
       exporter/
              metrics.py                      # Prometheus metric
definitions
       prometheus/
              prometheus.yml                  # Prometheus config
              alert_rules.yml                 # Alerting rules
       alertmanager/
              alertmanager.yml                # Alertmanager Slack
routing
       grafana/
              dashboards/
                  mlops_dashboard.json        # Exported Grafana
dashboard
       deploy/
              deploy.sh                       # AWS deployment
script
       tests/
              test_schema.py                  # Unit test: schema
change detection
              test_drift.py                   # Unit test: drift
detection
              test_predict.py                 # Unit test: /predict
 endpoint
       docker-compose.yml                   # Full observability
stack
       requirements.txt                     # Python dependencies
       .env.example                         # Example env vars (no
real secrets)
       README.md                            # Documentation
```

11. Submission Requirements

Submit the following by the deadline announced in class:

1. **GitHub Repository URL** — must be public (or shared with the instructor/TA). The repository must contain all source code as described in Section 8.
2. **README.md** — must include:
   - Team member name(s) and roll number(s).
   - Short description of the project.
   - Step-by-step setup and run instructions (local and AWS).
   - AWS EC2 public IP and instructions for testing `/predict` and `/metrics`.
   - How to configure the Slack webhook URL.

- Description of each alert and how to trigger it.

3. **Screenshots folder** (`screenshots/`) containing:
   - EC2 instance running and `/health` returning 200.
   - Raw `/metrics` output.
   - Grafana dashboard with live data.
   - All 7 Slack alert messages (one screenshot per alert).
   - GitHub Actions workflow with all jobs passing (green).
4. **Video Demo** (3–5 minutes): Screen recording demonstrating the full system: data ingestion, a firing alert, Grafana dashboard, and a passing CI/CD run. Upload to Google Drive or YouTube (unlisted) and include the link in your README.

12. Grading Rubric

#	Criterion	Marks	Part
1	**Data Ingestion & Schema Monitoring** Correct API polling; schema change detection (add/remove); 503 handling; drift detection logic.	15	Part 1
2	**ML Model Training** Classifier trained; accuracy ≥ 0.80 enforced; model serialized and versioned.	15	Part 2
3	**Auto-Retraining** Retrain triggered correctly on accuracy drop / drift / schema change; redeployment automated; Slack notification sent.	10	Part 2
4	**AWS Deployment** EC2 instance running; `/predict`, `/metrics`, `/health` endpoints publicly accessible; deployment script provided.	15	Part 3
5	**Prometheus Metrics** All 8 required metrics exposed correctly; metrics are live (not mocked).	10	Part 4
6	**Prometheus + Grafana + Alertmanager Stack** docker-compose runs all three services; Prometheus scrapes EC2; Grafana dashboard has all required panels; Alertmanager routes to Slack.	15	Part 5
7	**Alerts** All 7 alert rules defined; all alerts demonstrated firing in Slack with screenshots.	10	Part 6
8	**GitHub Actions CI/CD** Workflow file present; lint + test + build + deploy jobs all pass; screenshot of green run provided.	5	Part 7
9	**Documentation & Code Quality** README complete; code commented; no hardcoded credentials; project structure clean.	5	All
Total		**100**	
Bonus Marks (up to +10)			
B1	**Grafana Dashboard Excellence** Additional panels, annotations, or alert overlays on the dashboard.	+3	Part 5
B2	**Infrastructure as Code** EC2 provisioning automated using Terraform or AWS CloudFormation.	+4	Part 3
B3	**Extended CI/CD** Workflow includes automated model accuracy test gate (fails build if accuracy < 0.80).	+3	Part 7

13. Suggested Time Plan

The following is a suggested allocation to complete the project within the 8–9 hour window:

Phase	Task	Estimated Time	Marks
1	Data Ingestion + Schema + Drift Detection	1.5 h	15
2	Model Training + Auto-Retraining Logic	1.5 h	25
3	AWS EC2 Deployment + Dockerfile	1.5 h	15
4	Prometheus Metrics Exporter	0.5 h	10
5	Docker Compose Stack (Prom + Grafana + AM)	1.0 h	15
6	Slack Alerts Configuration	0.5 h	10
7	GitHub Actions CI/CD + Unit Tests	1.0 h	5
8	Documentation, README, Screenshots	0.5 h	5
Total		**8.0 h**	**100**

14. Academic Integrity

Important

All submitted work must be your own. You may consult online documentation, Stack Overflow, and AI tools for understanding concepts and debugging, but you may **not** copy complete solutions from other sources or from public GitHub repositories.
Plagiarism or sharing of complete solutions between students will result in a **zero** for all involved parties and may be escalated to the department for further action under FAST NUCES's academic integrity policy.

15. Helpful Resources

- **Prometheus Python Client:** `https://github.com/prometheus/client_python`
- **Alertmanager Slack Integration:** `https://prometheus.io/docs/alerting/latest/configuration/`
- **FastAPI Documentation:** `https://fastapi.tiangolo.com/`
- **Docker Compose Reference:** `https://docs.docker.com/compose/`
- **GitHub Actions Documentation:** `https://docs.github.com/en/actions`
- **Grafana Getting Started:** `https://grafana.com/docs/grafana/latest/getting-started/`
- **AWS EC2 Free Tier:** `https://aws.amazon.com/free/`
- **Evidently AI (optional drift library):** `https://www.evidentlyai.com/`

Good luck! Build something you'd be proud to put in your portfolio.

FAST NUCES — Machine Learning Operations, Spring 2026